```
     Failure/Error: expect(last_response.status).to eq(200)

       expected: 200
            got: 404

       (compared using ==)
« truncated »
```

Our app is returning a 404 (Not Found) status code. That's not surprising; we haven't added any routes to the Sinatra code yet. Go ahead and do so now:

04-acceptance-specs/03/expense_tracker/app/api.rb

```
post '/expenses' do
end
```

When you rerun your specs, you should see a passing result.

Filling In the Response Body

Think about what you'd like the response data to look like. It'd be nice to get back a unique ID for the expense we just recorded so that we can refer to it later. Let's return a JSON object that looks something like this:

```
{ "expense_id": 42 }
```

Ruby's JSON library can safely parse simple a record like this into a Ruby hash:

```
>> require 'json'
=> true
>> JSON.parse('{ "expense_id": 42 }')
=> {"expense_id"=>42}
```

At this point, we don't really care what the specific ID is, just that the data has this general structure. RSpec's matchers make it easy to express this idea. Add the following highlighted lines inside your spec, just after the HTTP response check:

04-acceptance-specs/04/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb

```
it 'records submitted expenses' do
  coffee = {
    'payee'  => 'Starbucks',
    'amount' => 5.75,
    'date'   => '2017-06-10'
  }

  post '/expenses', JSON.generate(coffee)
  expect(last_response.status).to eq(200)

➤ parsed = JSON.parse(last_response.body)
➤ expect(parsed).to include('expense_id' => a_kind_of(Integer))
end
```